

Frosch Bottle Inspection

System Architecture & Sequence Diagrams

Task INTER-54 | Evaluation package

Engineer: Noman | Lead: Abdul Moiz

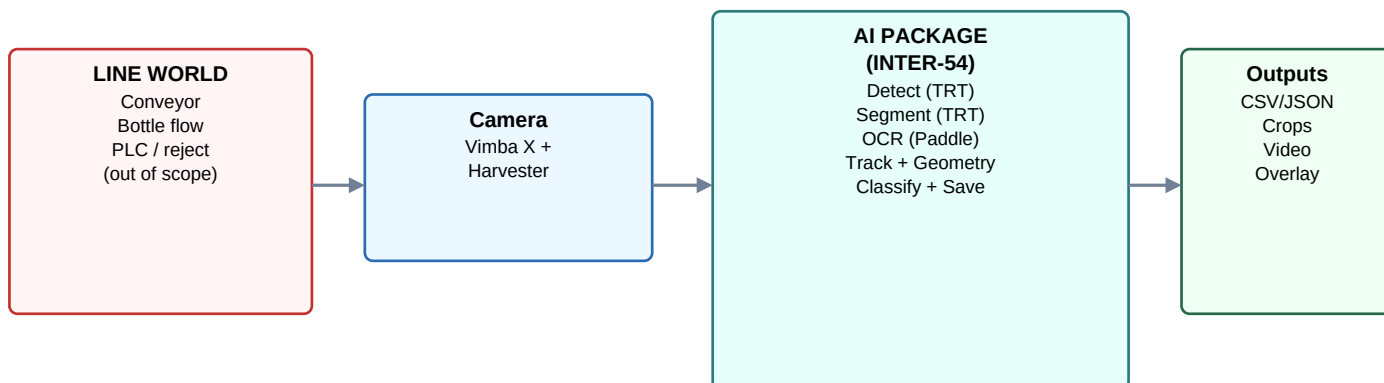
Contents

1. System context (boundaries)
2. End-to-end pipeline architecture
3. Runtime software structure (src / configs)
4. Sequence diagram — one bottle (live path)
5. Classification decision flow
6. Training & model export flow
7. Data & artifact traceability map
8. Failure / fallback behaviour map

Scope: only Frosch INTER-54 components (camera, RF-DETR det/seg, TensorRT, PaddleOCR, tracking, geometry rules, configs, outputs). No unrelated products or generic templates.

1. System context and boundaries

What is inside the AI package vs what is outside (line integration).



In scope

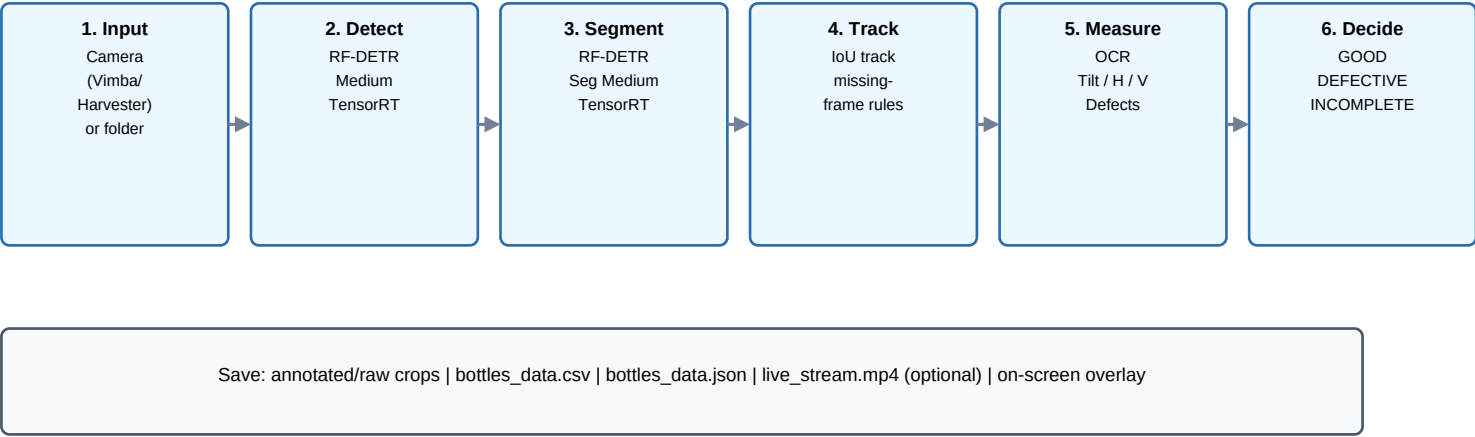
- Live camera and folder input modes
- RF-DETR Medium detection + RF-DETR Seg Medium (TensorRT)
- PaddleOCR capacity read (100/300/500 ml)
- Tracking, tilt, H/V, defect confirmation
- GOOD / DEFECTIVE / INCOMPLETE + file outputs
- configs/*.yaml thresholds and paths

Out of scope

- PLC / mechanical reject hardware
- Contact mm gauge metrology product
- Customer UI packaging beyond overlay + files
- Training data stored inside Git

Bottle sizes in scope: 100 ml, 300 ml, 500 ml

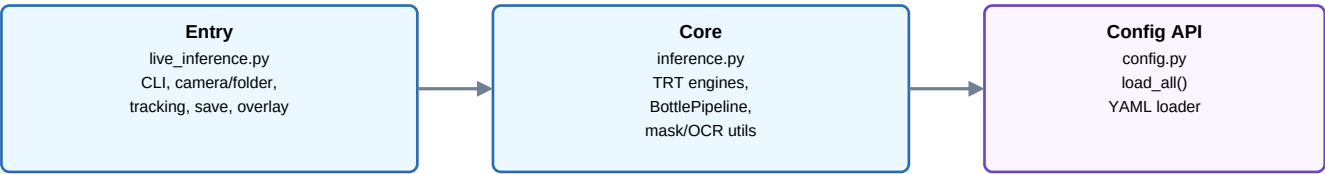
2. End-to-end pipeline architecture



Stage responsibilities

Input	Acquire frame; camera CTI from configs/camera.yaml or load folder images
Detect	Bottle, label, capacity, bump, damage, scratch with class confidence thresholds
Segment	Bottle/label masks for geometry and defect overlap
Track	Associate detections across frames; trigger-line / max missing frames finalisation
Measure	PaddleOCR capacity; PCA tilt; H/V vs expected V(capacity); defect streak + overlap
Decide	Apply A02 rules; emit GOOD / DEFECTIVE / INCOMPLETE; write outputs

3. Runtime software structure



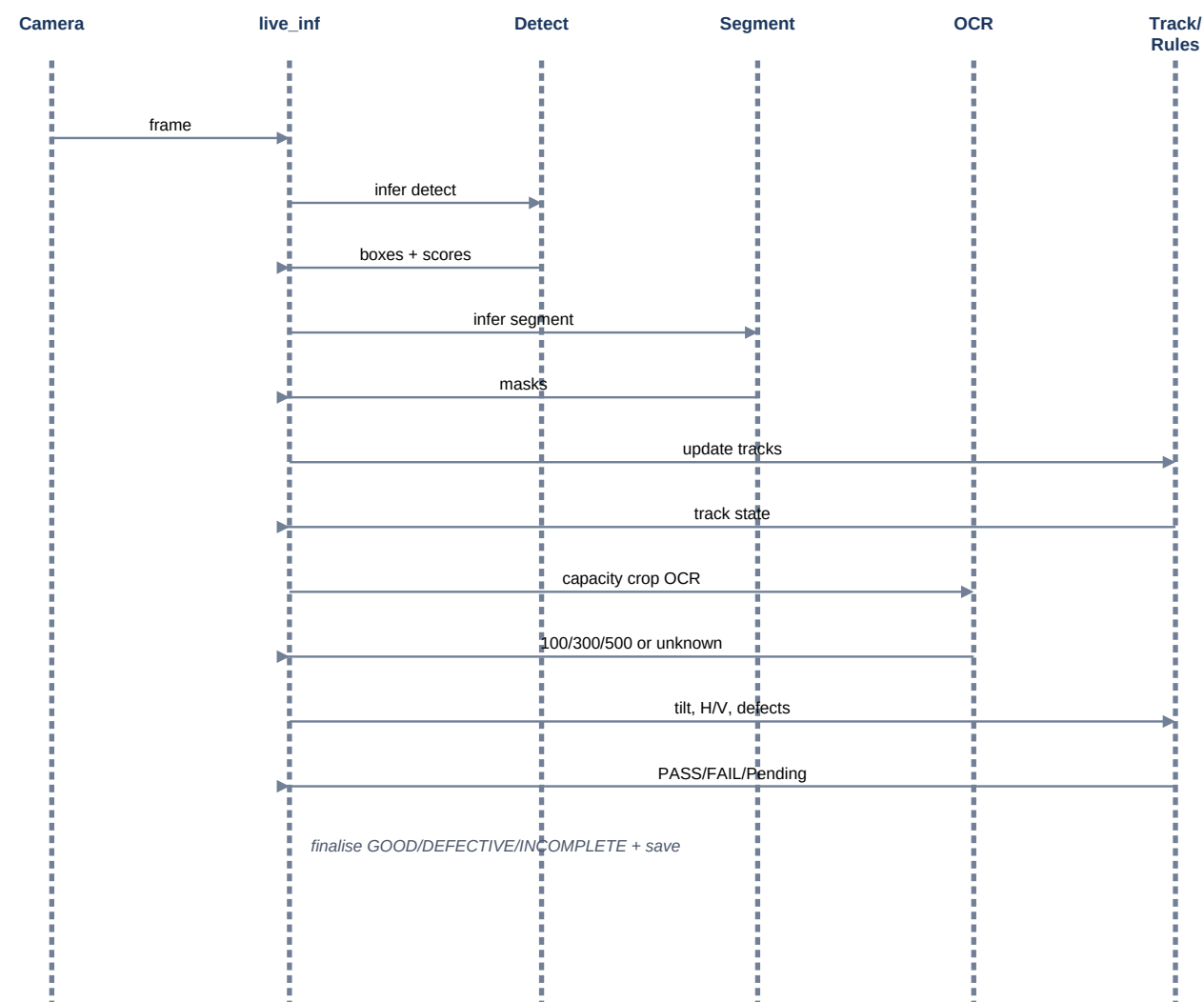
configs/ (thresholds and paths — not hardcoded)

detection.yaml	engine path, class confidence thresholds
segmentation.yaml	seg engine, score threshold
ocr.yaml	GPU flag, min conf, known capacities, crop pads
geometry.yaml	tilt, H/V tolerances, expected V by capacity
tracking.yaml	IoU, missing frames, defect streak, trigger line
pipeline.yaml	save paths, csv/json/video names
camera.yaml	CTI path, device index

Supporting tree

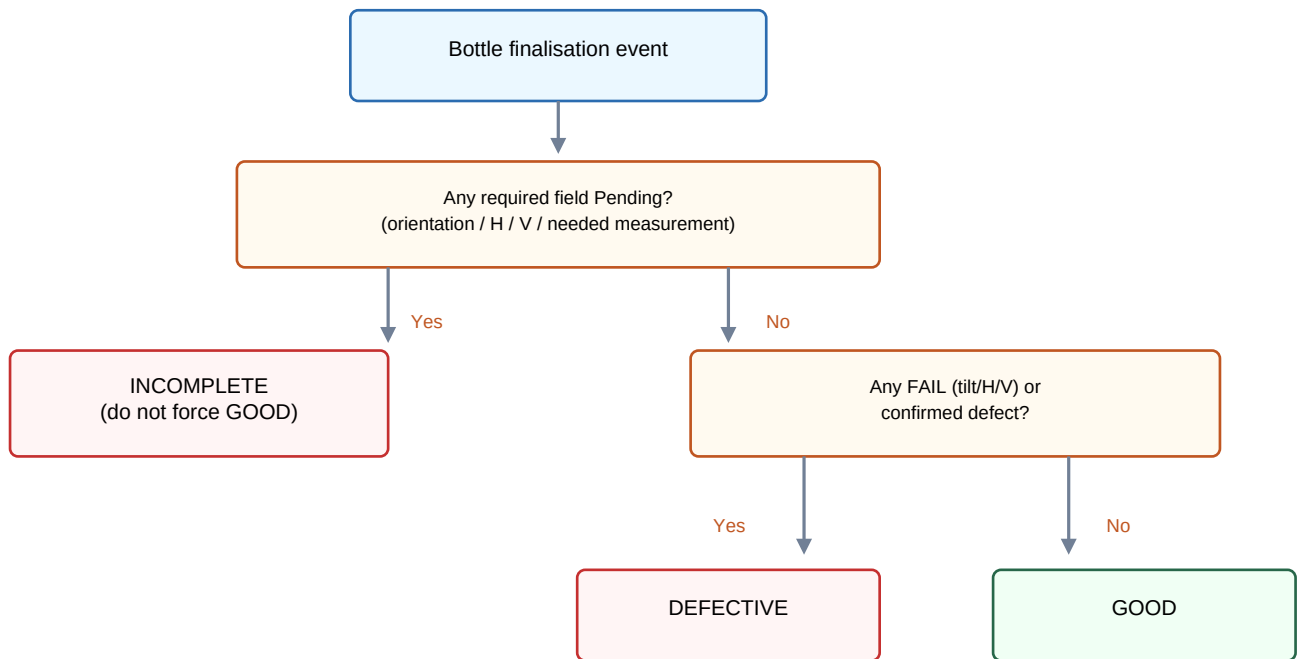
- tests/ — contract tests (config, postprocessing, code review, evidence) + quality scripts
- docs/artifacts/ — A01–A17 handbook records
- results/ — sample bottle CSV/JSON outputs
- Model store (not in Git) — .pth checkpoints, .trt engines, dataset frosch-bottle-5-ypv4gi

4. Sequence — one bottle on live path



Finalisation: trigger-line and/or max missing frames. Pending measurements → INCOMPLETE (not GOOD).

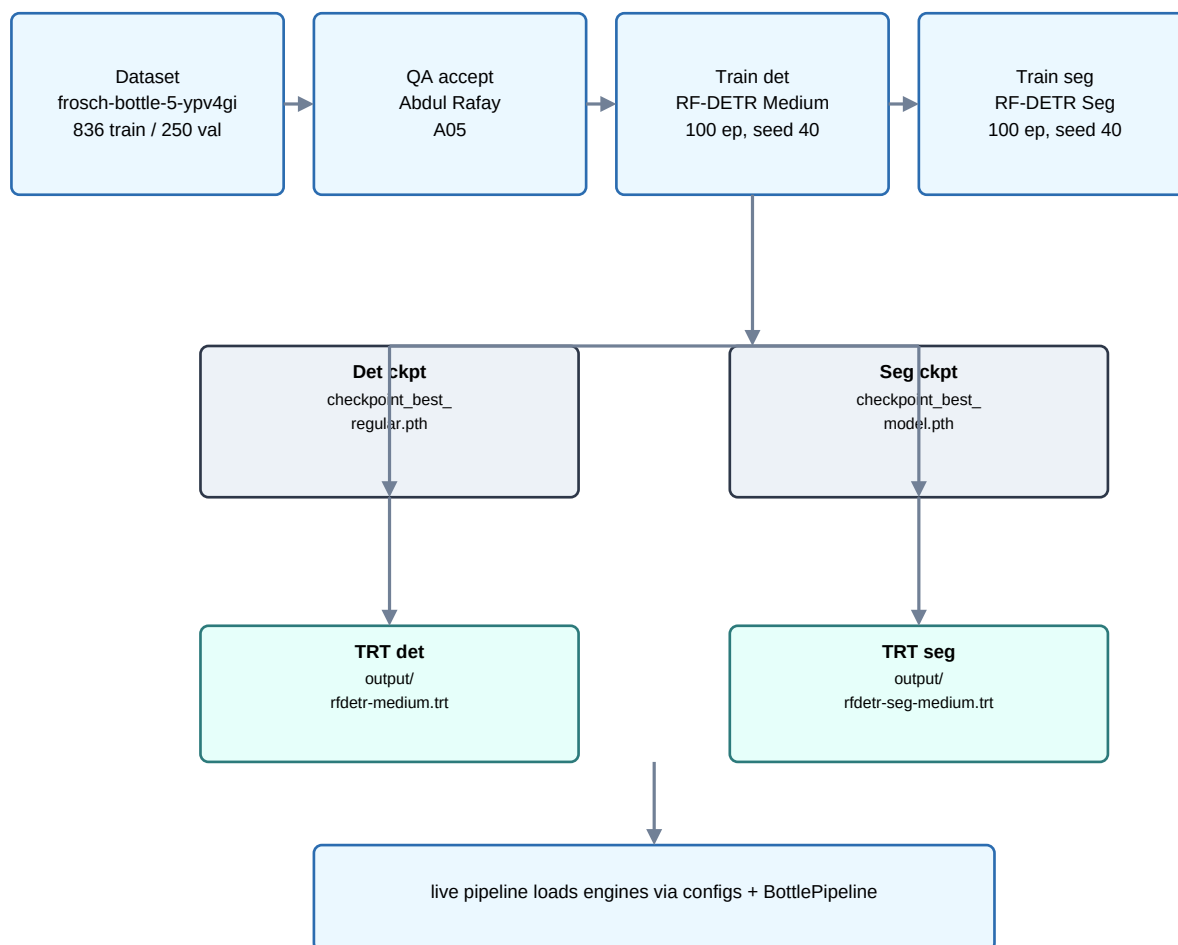
5. Classification decision flow



Rule summary (from A02 / configs)

- $\text{Tilt} \leq 45^\circ \rightarrow \text{orientation PASS}; \text{else FAIL}$
- $|H| \leq 0.15 \rightarrow \text{H PASS}; \text{else FAIL}$
- $|V - \text{expected_V}(\text{capacity})| \leq 0.05 \rightarrow \text{V PASS}; \text{else FAIL}$
- expected_V depends on capacity 100 / 300 / 500 ml (geometry.yaml)
- Defect confirmed only with mask overlap ≥ 0.30 and confirmation streak
- Capacity OCR must resolve to known set or remain unknown (no false pass)

6. Training and model export flow



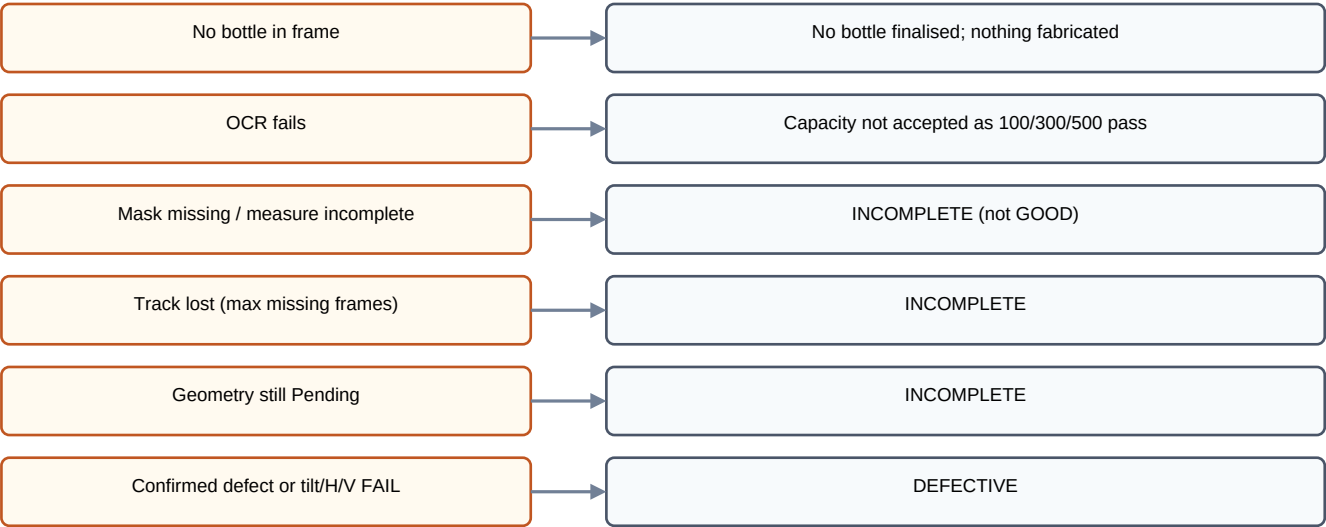
Training notes: LR 0.001, batch 8, no augmentation, input 576×576, no early stopping.

Weights/engines stay in model store (not Git). Dataset referenced by ID only.

7. Artifact / code traceability (handbook path)

- A01 QS → A02 Objective (Lead: Abdul Moiz)
- A03 Inventory → A04 Schema → A05 QA (Abdul Rafay) → A06 Dataset version
- A06 → A07 Det run + A07 Seg run → A17 metrics
- A02 + A07 → A08 Evaluation → A09 GT sample validation
- A08/A09 → A11 Soak (1009 completed) + A12 Fallback audit → A13 Evaluation manifest
- Code path: configs/*.yaml → src/config.py → inference.py / live_inference.py → results/

8. Failure / fallback behaviour map



Handbook rule reflected here: Pending/missing measurement must not be silently converted to GOOD.